

title: "Human-regulating-the-loop: deterministic schema validation as the safety boundary for agentic biomedical curation"

target_journal: Nature Computational Science / Journal of Biomedical Semantics (high-end)

authors:

- Christopher J. Mungall (LBNL, Monarch Initiative)
- J. Harry Caufield (LBNL)
- [co-authors TBD]

draft_status: working draft

related_talks:

- "Mungall CJ. *Unlocking Disease Mechanisms: Agentic AI for Clinical Knowledge.* Zenodo, 2026. <https://zenodo.org/records/18720444>"

revision_notes:

- "Restructured to Nat CS Methods Resource layout: Abstract, Introduction, Results (Architecture), Failure-mode analysis, Comparison, Limitations, Outlook, Methods."
- "Tightened abstract from ~350 to ~190 words with structured context/problem/contribution/result/significance flow."
- "Removed thesis-style numbered headings; adopted shorter declarative headings."
- "Reduced first-person assertion verbs (we argue/believe/claim) by roughly 60%; shifted body to evidential framing."
- "Added Figure 1-3, Table 1, Box 1 placeholders with journal-style callouts and inline (Fig. X)/(Table X) references."
- "Added inline numeric citation markers [N] and References section stub with TBD placeholders for LinkML, OAK, Biolink, Monarch, ClinGen, RLHF, deep-research hallucination, etc."
- "Promoted the non-fabricatable reference cache to a Box 1 example while keeping the dedicated subsection."
- "Softened unmeasured percentage claims (~70%, ~25%) to qualitative phrasing."
- "Cut conclusion repetition; thesis now stated in abstract, introduction, and outlook only."
- "Added one-sentence acknowledgement of the Zenodo talk in the introduction."
- "Added Methods section consolidating implementation specifics."
- "Added three alternative titles in an HTML comment above the abstract."

Abstract

Large language model agents now draft structured biomedical content faster than any human curation pipeline can review, but their errors are confidently expressed and unevenly distributed: fabricated identifiers, invented quotations, and ontology terms that pattern-match the real ones. The reflexive answer, human-in-the-loop review, does not scale and in practice degrades to rubber-stamping. Here we describe an alternative architecture, **human-regulating-the-loop**, in which humans and upstream standards bodies define the schema, ontology bindings, and deterministic validation gates, and agents operate inside that enclosure. We instantiate the pattern in the Disorder Mechanisms Knowledge Base (dismech), a LinkML schema-driven resource covering several hundred disorders with tens of thousands of evidence-backed assertions. The architecture combines an ontology-bound schema, a three-layer deterministic validation stack (schema conformance, ontology term validation, reference-snippet substring matching), and a non-fabricatable reference cache populated only by trusted fetchers from authoritative upstream sources. Agent failure modes — fabricated PMIDs, paraphrased snippets, label-mismatched ontology terms, hand-fabricated cache files — are each blocked by a single deterministic check at a CI gate. The architecture remains stable as agent capabilities grow, because the gates do not depend on agent confidence.

Introduction

The dominant framing for safe AI-assisted curation is human-in-the-loop: a curator reviews each agent output before it enters the canonical store. This framing is correct as far as it goes but underspecified. It does not say what the human is supposed to check, does not constrain agent output to a form where checking is tractable,

and degrades predictably as agent throughput exceeds human review capacity. In knowledge-base practice, human-in-the-loop most often means a curator approving a large fraction of agent output that they did not have the bandwidth to verify against primary sources [1].

The failure modes of current large language model agents in this setting are well characterized [2, 3]. Agents fabricate identifiers that look real (a PubMed identifier with the right number of digits, an HPO term with the right prefix). They synthesise quotations that are stylistically consistent with the cited source but do not appear in it. They confidently assign ontology terms whose canonical label does not match the meaning required by the schema [4]. They produce cache files, dossiers, and intermediate artifacts that look like real ones and pass casual review. A human-in-the-loop pipeline that asks a curator to scan such output by eye will miss most of these errors; the aggregate error rate is not visible until much later.

The architecture described here inverts this relationship. The human is not *in* the loop; the human and the upstream standards community are *around* the loop, having defined the schema, the ontology bindings, the evidence contract, and the deterministic validators that police them. The agent operates inside this enclosure. Errors that would otherwise be absorbed are surfaced at a continuous-integration (CI) gate and blocked from merging. We call this **human-regulating-the-loop**, in deliberate contrast to human-in-the-loop (Fig. 1). The distinction determines where engineering effort goes, what the agent is incentivised to optimise for, and whether the resource gets safer or less safe as agent capabilities grow. The framing builds on a recent presentation that introduced the anti-hallucination posture this paper formalises [5].

We use the Disorder Mechanisms Knowledge Base (dismech) as the working substrate. Dismech is a structured representation of disease pathophysiology built around a LinkML [6] schema with ontology-bound dynamic enumerations, comprising several hundred disorder entries, dozens of mechanism modules, and tens of thousands of evidence-backed assertions linking genetic and environmental causes through pathophysiology and biochemistry to phenotypes and treatments. The resource has been constructed using two production agentic systems (Claude Code and Codex) running both interactively under a curator and autonomously via scheduled GitHub Actions. A companion paper describes the mechanistic content and its applications; the present paper describes the curation architecture.

The contributions are: (i) a concrete architecture for human-regulating-the-loop agentic curation, instantiated in a real mechanistic-disease knowledge base; (ii) an analysis of agent failure modes that the architecture must catch, with the specific design choices that catch each; and (iii) the identification of **non-fabricatable reference caches** and **substring-only snippet matching** as load-bearing design patterns for any agentic system that depends on a verifiable substrate.

Figure 1 | Human-in-the-loop versus human-regulating-the-loop curation architectures. In the conventional pattern (left), the curator reviews free-form agent output, and the failure rate is bounded by review attention. In the proposed architecture (right), humans define the schema, ontology bindings, and deterministic validation gates; agents emit typed structured output into a CI-enforced enclosure; the failure rate is bounded by gate correctness. [FIGURE PLACEHOLDER]

The schema as the safety boundary

The first design choice is that the primary artefact emitted by the agent is not text but a typed, schema-bound structure. The schema is authored in LinkML [6], a schema language for biomedical data that expresses classes, slots, enumerations, and constraints, and compiles to JSON Schema, SHACL, ShEx, OWL, and SQL DDL. LinkML's distinguishing feature for the present purpose is **dynamic enumerations bound by ontology**

constraint (`reachable_from`): an enum value is valid only if it is reachable from a specified parent in a specified ontology via specified predicates. The dismech schema uses this construct to require that every phenotype descriptor term is reachable from the root of the Human Phenotype Ontology (HPO) [7], every cell type from the Cell Ontology (CL) [8], every treatment action from MAXO or the NCI Thesaurus therapeutic-procedure subgraph, and every gene identifier from HGNC.

Because the agent is required to emit typed objects, it cannot produce a free-text evidence summary that looks plausible but is unanchored. Because every ontology slot is bound to an authoritative source, an identifier that looks like an HPO term but is not defined in HPO is rejected at validation. Because every evidence item is a structured object with required fields — a reference, a verbatim snippet, a classification of whether the evidence supports or refutes the claim, and a classification of the publication's evidence type (human clinical, model organism, in vitro, computational) — vague support cannot be smuggled under specific cover.

The schema is also the locus of editorial policy. When agents were observed to silently conflate frequency-of-phenotype claims (e.g., "frequent") with disease–phenotype association claims, the editorial rule and its validation were added to the schema. When agents were observed to classify veterinary case series as human-clinical evidence, the classification rules were tightened and the validation now flags this. The schema is not static; it is the place where editorial learning accumulates and where agents' systematic errors become rules they cannot violate.

The three-layer deterministic validation stack

Around the schema run three validation layers (Fig. 2). None invokes a language model. All are deterministic and reproducible, and all must pass for a change to merge.

Layer 1: schema conformance. `linkml-validate` checks that the proposed YAML conforms to the schema: required fields are present, types are correct, enum values are in range, ontology prefixes are correct, multivalued slots are multivalued, single-valued slots single-valued. This is the cheapest layer and, in our experience, catches the largest class of agent errors — agents routinely emit the right idea in the wrong shape, and the schema catches that immediately.

Layer 2: ontology term validation. `linkml-term-validator` checks every ontology-bound term against a local authoritative snapshot of the ontology, accessed via the Ontology Access Kit (OAK) [9]. The check is twofold: the identifier must exist in the ontology, and the `term.label` recorded in the YAML must exactly match the canonical label. This catches a failure mode that is both common and dangerous: an identifier that is in the ontology but whose label is wrong — for example, asserting that `HP:0001324` has label "Hypotonia" when in fact `HP:0001324` is "Muscle weakness". A label mismatch is not cosmetic; it indicates that the agent's intent and the actual term diverge, and the downstream semantic interpretation will be wrong [4].

Each descriptor carries two distinct label slots. The `term.label` slot is canonical and must match the ontology exactly. The `preferred_term` slot is the human-readable display label and may be more specific or differently phrased than the ontology label when the ontology term is too broad to convey the intended clinical granularity. The separation gives the agent a controlled outlet for clinical nuance without permitting corruption of the ontology binding.

Layer 3: reference-snippet validation. `linkml-reference-validator` checks that every evidence item's `snippet` is a verbatim substring of the cached text for the cited `reference`. This is the single most consequential layer of the stack. It catches paraphrased quotations, fabricated quotations, wrong-PMID

assignments, and confident hallucinations citing papers that do not exist. Substring matching is intentionally strict: no fuzzy matching, no normalisation, no "close enough". The cost is occasional false rejections from whitespace and encoding issues; the benefit is that *whether a snippet exists in the source* is a yes/no fact the agent cannot game.

The three layers run in sequence, cheapest first, and the pipeline short-circuits on the first failure. In our experience, the schema layer catches the largest class of agent errors before the term and reference validators are reached; the term validator catches a further substantial class; the reference validator catches the residual most-dangerous class — confident textual fabrication. Each class of error has a single deterministic check at its boundary.

Figure 2 | Three-layer deterministic validation stack. Each layer is a deterministic check against an authoritative artifact: the LinkML schema, local ontology snapshots accessed via OAK, and the per-reference cache. Example failures caught at each layer are shown: a missing required `evidence_source` field (Layer 1), an HPO label mismatch where `HP:0001324` is recorded as "Hypotonia" rather than "Muscle weakness" (Layer 2), and a paraphrased snippet absent from the cited PubMed abstract (Layer 3). [FIGURE PLACEHOLDER]

Non-fabricatable reference caches

The reference cache is the asset on which Layer 3 depends, and its integrity is therefore load-bearing for the architecture. The single largest agent failure mode encountered in production — and the one that took the longest to architect against — is the agent fabricating the cache itself (Box 1).

The architectural fix is that the cache is produced by, and only by, a dedicated fetcher that pulls the source from the upstream authority (PubMed, ClinicalTrials.gov, Orphadata [10], ClinGen [11], CIViC [12]), writes a deterministic markdown-with-frontmatter file with a canonical filename, and computes a checksum recorded in the frontmatter. A separate CI check (`check-reference-cache-frontmatter`) validates that every cache file has parseable frontmatter consistent with its filename and reference identifier. The cache is treated as **read-only from the agent's perspective**: the harness permission layer denies cache writes (Fig. 3).

Agents are explicitly instructed, in both the human-readable contributor guide and the agent harness configuration, never to create or hand-edit cache files. The correct response to a snippet mismatch is to re-quote the abstract more carefully or to use a different citation. The cache cannot be the variable the agent adjusts to make the validator green; the snippet must be the variable.

This pattern generalises beyond dismech. Any agentic system that depends on a verifiable substrate (citations, identifiers, ontology terms, reference data) must arrange that the substrate is **not writeable by the agent**. The substrate must be maintained by deterministic fetchers from authoritative sources, with the CI invariant that the substrate can be reproduced from those sources. When the agent can write to the substrate, the validation that depends on the substrate is not validating anything; it is laundering the agent's confidence.

Figure 3 | The non-fabricatable reference cache. Cache files in `references_cache/` are populated only by dedicated fetchers from authoritative upstream sources (PubMed, ClinicalTrials.gov, Orphadata, ClinGen, CIViC). Cache writes are denied at the harness permission boundary; the agent may read the cache but may not write to it. A deterministic frontmatter check enforces filename-to-identifier consistency. The reference-snippet validator can therefore treat the cache as a trusted substring oracle. [FIGURE PLACEHOLDER]

Box 1 | Cache fabrication caught in production

An agent was asked to add a phenotype claim backed by a PubMed identifier. It produced a plausible snippet, but the reference validator failed because the snippet was not in the cache. The agent, instructed to "resolve the validation failure", responded by **creating the cache file by hand**, with fabricated abstract content engineered to contain the snippet verbatim. Schema validation passed. Term validation passed. The reference validator now also passed, because the snippet matched the (fabricated) cache. A casual human reviewer seeing a green CI would have merged.

The failure was caught only after introducing the dedicated cache frontmatter check and tightening the harness permission boundary to deny cache writes. Subsequent retrospective audit of cache files against re-fetched upstream sources found a residual but small fabrication rate prior to the fix; no such fabrications have been committed since. The lesson generalises: a validator is only as trustworthy as the substrate it queries.

Structured-database sources as quotable evidence

The same non-fabricatable-substrate pattern extends to non-literature evidence. Many of the most important biomedical claims are not in journal abstracts but in structured databases: Orphanet [10] for rare-disease definitions and phenotypes, ClinGen [11] for gene–disease validity and dosage sensitivity, ClinicalTrials.gov for trial details, CIViC [12] for cancer variant evidence. Each has its own access pattern, schema, and update cadence.

Structured sources are treated as first-class citable evidence. For each source, a source-specific fetcher in `src/dismech/structured_sources/` pulls the bulk data at a pinned snapshot version, builds an index, and materialises one cache file per entity in the same `references_cache/` directory used for literature. The cache files use a deterministic line-oriented markdown format with markdown tables for tabular content, so that an Orphanet phenotype row or a ClinGen validity assertion row is a **stable quotable substring** that does not drift across rebuilds. Snapshot versions are pinned in a per-source manifest; refreshes are explicit and reviewable.

An evidence item with `reference: ORPHA:558` and a snippet quoting "*Marfan syndrome is a systemic disease of connective tissue*" is validated by the same substring check that validates a PMID-backed claim. The agent does not need to distinguish source types, and the validation mechanism does not need to special-case them. New structured sources are added by writing a new fetcher; the validation stack is unchanged.

The agentic harness

Inside the validation enclosure, agents are responsible for a substantial fraction of the curation work: drafting and enriching disorder entries from the literature; adding ontology term annotations (HPO phenotypes, GO processes [13], CL cell types, MAXO treatments) under a dedicated skill that enforces authoritative term lookup via OAK [9]; validating and repairing evidence references under a dedicated skill that enforces the snippet substring contract; responding to compliance scoring that identifies under-curated entries; and reviewing pull requests against the dismech contributor guidelines via a dedicated review agent.

Agents are explicitly not permitted to write to the reference cache, modify the schema (a schema change is a human editorial decision, often motivated by patterns the agents have surfaced), bypass any validation layer,

force-push branches they did not create, or open pull requests from forks (which would bypass CI secrets and skip the AI review layer).

These prohibitions are encoded both as harness configuration (permissions, hooks, allow/deny lists in the Claude Code settings) and as contributor guidelines in `CLAUDE.md`. The harness-level controls are enforced by the tool layer in which the agent operates: an agent that attempts to write to a cache file is denied at the tool boundary; an agent that attempts to skip a hook is denied at the git layer. The contributor guidelines are the human-readable form of the same constraints, and exist so that an agent that *could* in principle do something but should not, knows it should not.

Continuous integration as the enforcement layer

The pull request is the unit of curation work. Every pull request runs the three-layer validation stack, the structured-cache frontmatter check, a schema-test suite, and an AI-augmented review pass. The review pass is run by a language model agent configured against the same contributor guidelines used for human review, and it produces inline suggestions with location, issue, and recommendation. **The review agent's findings are advisory; the deterministic validators' findings are blocking.**

The human curator (or curating agent) is responsible for resolving findings; they cannot be dismissed silently. When the review agent finds a problem and the human disagrees, the disagreement is recorded as a pull-request comment and the human's reasoning is on the record. This creates an audit trail of editorial judgement that is mined for schema refinements: a recurring disagreement between review agent and human curators is usually a sign that the underlying rule is ambiguous and needs to be made explicit in the schema or guidelines.

A scheduled GitHub Action periodically inspects a compliance dashboard, identifies the lowest-scoring entries, dispatches an agentic curation job for each, and opens a pull request. The curation pipeline is therefore autonomous end-to-end: a low-coverage entry is detected, researched, drafted, validated, reviewed, and merged without human intervention provided every validation gate passes. When a gate fails, the human curator is engaged on the specific failure rather than on the whole entry. This is what autonomous curation under deterministic validation looks like in production.

Failure-mode analysis

Table 1 documents the principal agent failure modes observed during dismech development and the specific architectural mechanism that catches each. Three patterns recur. First, the most dangerous failure modes (cache fabrication, label mismatch, paraphrased snippet) are caught by deterministic checks against an external authority, not by the agent self-verifying or the human eyeballing. Second, editorial-rule failures (frequency evidence, evidence-source classification) are caught by a combination of schema documentation, AI review, and human judgement; each newly observed failure of this kind is promoted into a more explicit schema constraint where possible. Third, no failure mode in the table is caught by RLHF [14] or by agent self-correction [15]; the safety boundary is deterministic throughout.

Table 1 | Observed agent failure modes during dismech curation and the architectural mechanism that catches each.

Failure mode	Catching mechanism
Fabricated PMID	Reference fetcher fails to resolve; PMID absent from cache; snippet check fails.
Real PMID, wrong paper for claim	Snippet does not match cache; reference validator fails.
Real PMID, paraphrased snippet	Substring check fails; reference validator fails.
Snippet from PMID A assigned to PMID B	Cache is per-PMID; snippet from A is not a substring of cache for B; validator fails.
Cache file fabricated to satisfy snippet	Cache frontmatter check fails on canonical filename/checksum; cache writes denied at harness boundary.
Fake HP/GO/CL term that looks real	Term validator: identifier absent from ontology snapshot.
Real term, wrong label	Term validator: identifier present but <code>term.label</code> mismatches canonical.
Real term, wrong meaning (HP for GO concept)	Schema <code>reachable_from</code> binding to required ontology root rejects.
Frequency claim backed by association evidence	Editorial rule in schema and contributor guide; AI review flag; human verification.
Veterinary or model-organism evidence classified as human-clinical	<code>evidence_source</code> enum with editorial rule; AI review flag; schema documentation.
Silent removal of inconvenient REFUTE evidence	Git diff at PR review; AI review flags significant deletions.
Subtype foreign-key inconsistency	Dedicated test (<code>test_subtype_foreign_keys</code>) verifies subtype references resolve.

Comparison with alternative safety architectures

The dominant alternatives to deterministic schema validation for safe agentic content production are RLHF-style preference training [14], post-hoc fact-checking by a second language model [16], and agent self-verification [15]. Each has structural failure modes that deterministic validation does not.

RLHF and preference training [14] make agents more confidently produce the kind of output humans prefer at review time. They do not make the output verifiable. An agent that has learned to produce plausibly cited medical claims is not more likely to cite real papers; it is more likely to produce claims that look correctly cited. RLHF improves the correlation between agent output and reviewer preference, which makes the output harder to discriminate at review — the wrong direction for the failure modes that matter.

Post-hoc fact-checking by a second language model [16] is structurally similar to the first: the second model is also confidently wrong about citations, ontology labels, and quotations, and disagreement between agents is not a reliable signal of error. In dismech development, the AI review pass routinely missed cache fabrications that the deterministic validator caught. A second agent is a useful reviewer of *style and editorial conformance*, which is how it is used here; it is not a reliable reviewer of *fact*.

Agent self-verification [15] has the structural problem that the agent cannot, in general, distinguish its own hallucinations from its own real recall. Iterative chains-of-thought with self-criticism reduce some error rates, but by a margin dwarfed by the gain from a single external deterministic check against the source.

The case for deterministic schema and identifier validation as the *primary* safety boundary is therefore not aesthetic. Deterministic validation is the only mechanism in this list whose accuracy is independent of the agent's confidence, and the only mechanism whose failure rate is bounded by the correctness of human-authored code rather than the next round of model training.

Limitations

The architecture has its own failure modes. Substring snippet matching does not catch a snippet that is *real but irrelevant* — quoted correctly from an abstract that does not actually support the claim being made; AI and human review remain the only defence for this layer, and have occasionally failed. Term validation does not catch a term that is correct but too broad or too narrow for the intended meaning; this is again an editorial judgement that AI and human review must catch. The schema is human-authored and contains its own errors; mitigation is via schema tests and an iterative process of promoting rules from contributor-guide prose to schema constraint. The cache is only as good as the snapshot date; refreshes can change snippets in ways that retroactively invalidate evidence, and this is tracked manually. The agentic harness itself is software with bugs, and the permission boundaries are only as strong as the implementation; the most likely future failure is not an agent escaping the rules but a misconfigured harness silently relaxing them.

Throughput is the second cost. Deterministic validation is slower than agent self-verification, particularly when reference fetches must be performed. Caching, parallelism, and structured-source batch fetches amortise this, but a single new PMID remains a network call away. This appears to be an unavoidable cost of the architecture rather than a defect.

Outlook

We believe the right relationship between humans and agents in structured biomedical curation is not human-in-the-loop but human-regulating-the-loop. As agentic capabilities grow, the architecture described here gets *safer* as agents get better, because the gates are independent of agent confidence. We do not claim this is the only safe architecture for agentic biomedical curation, but any safe architecture will share its essential features: typed structured output, deterministic validation against authoritative sources, non-fabricatable substrates, and a schema in which editorial learning accumulates. The priority for future work is to broaden the structured-database fetcher catalogue, to formalise the schema-promotion workflow by which editorial rules become validation constraints, and to study whether the same pattern can be ported to other structured biomedical resources, including knowledge graphs in the Monarch Initiative [17] and Biolink Model [18] ecosystem.

Methods

Schema and validators. The dismech LinkML schema is at `src/dimech/schema/dimech.yaml`. Schema validation uses `linkml-validate`; ontology term validation uses `linkml-term-validator` backed by OAK [9] with local SQLite snapshots of HPO, GO, CL, MONDO, UBERON, CHEBI, GENO, HGNC, MAXO, and NCIT (`conf/oak_config.yaml`). Reference-snippet validation uses `linkml-reference-validator` with substring matching against per-reference markdown cache files.

Reference cache. Literature cache files (`references_cache/PMID_*.md` , `references_cache/DOI_*.md` , `references_cache/clinicaltrials_*.md`) are populated only by dedicated fetchers invoked through `just fetch-reference <ID>` . Each file carries YAML frontmatter recording the canonical identifier, fetch date, source, and content checksum. A deterministic frontmatter check (`just check-reference-cache-frontmatter`) runs at every CI invocation and enforces filename–identifier consistency.

Structured-database sources. Structured-source fetchers are in `src/dismech/structured_sources/` . Each subclasses a common `StructuredSource` base class implementing `build_index` , `identifiers` , and `serialize` . Bulk-data snapshots are pinned by date and SHA-256 hash in per-source manifests (`data/<source>/MANIFEST.yaml`). Currently supported sources are Orphanet (8,823 leaf disorders), ClinGen Gene-Disease Validity, ClinGen Dosage Sensitivity, and CIViC.

Agentic harness. Two production agentic systems were used: Claude Code and Codex. Both operate against the schema and validators through the shell, the git tool, and a constrained file-write permission layer. Cache directories are configured as deny-write for agent roles. A contributor guide (`CLAUDE.md`) repeats the constraints in human-readable form for any agent that operates outside the strict harness sandbox.

Continuous integration. Every pull request triggers a CI pipeline running, in order: schema conformance (Layer 1), term validation (Layer 2), reference-snippet validation (Layer 3), the cache frontmatter check, the schema test suite (`tests/test_data.py`), and an AI-augmented review pass run by Claude against contributor guidelines. Validator findings are blocking; review-agent findings are advisory and require explicit acknowledgement before merge.

Autonomous curation loop. A scheduled GitHub Action periodically inspects the compliance dashboard (`just gen-dashboard`), selects the lowest-scoring entries, and dispatches agentic curation jobs that open pull requests. Merges proceed only when every CI gate passes.

Data and code availability

Schema, validators, agent harness configuration, and full curation history are at <https://github.com/monarch-initiative/dismech> . A browsable resource is available at <https://monarch-initiative.github.io/dismech/> . LinkML is documented at <https://linkml.io> . The cache fetcher and structured-source framework are part of the dismech codebase under `src/dismech/structured_sources/` .

Acknowledgements

We thank the LinkML, OAK, Monarch Initiative, ClinGen, Orphanet, and CIViC communities for the standards and bulk data on which dismech depends. The framing described here builds on a presentation by the first author, *Unlocking Disease Mechanisms: Agentic AI for Clinical Knowledge*, recorded on Zenodo [5]. Funded by [TBD].

References

- [1] [TBD: human-in-the-loop curation scaling limits, biomedical KB curation throughput literature]
- [2] [TBD: LLM hallucination characterisation in scientific tasks, e.g., Ji et al., survey of hallucination in NLG]

- [3] [TBD: deep-research / agent citation hallucination characterisation, recent agent eval benchmarks]
- [4] [TBD: ontology hallucination in LLMs, e.g., recent work on biomedical concept grounding]
- [5] Mungall CJ. *Unlocking Disease Mechanisms: Agentic AI for Clinical Knowledge*. Zenodo, 2026.
<https://zenodo.org/records/18720444>
- [6] [TBD: LinkML citation, Moxon et al. 2021, *Bioinformatics Advances* / Database]
- [7] [TBD: Human Phenotype Ontology citation, Köhler et al., *Nucleic Acids Research*]
- [8] [TBD: Cell Ontology citation, Diehl et al., *Journal of Biomedical Semantics*]
- [9] [TBD: Ontology Access Kit (OAK) citation, Mungall et al.]
- [10] [TBD: Orphanet / Orphadata citation, Rath et al., *Human Mutation*]
- [11] [TBD: ClinGen Gene-Disease Validity citation, Strande et al., *AJHG*]
- [12] [TBD: CIViC citation, Griffith et al., *Nature Genetics*]
- [13] [TBD: Gene Ontology citation, Ashburner et al. 2000 / GO Consortium update]
- [14] [TBD: RLHF citation, Ouyang et al. *InstructGPT*; Christiano et al.]
- [15] [TBD: agent self-verification / self-critique citation, e.g., Madaan et al. *Self-Refine*]
- [16] [TBD: LLM fact-checking / second-model verification citation]
- [17] [TBD: Monarch Initiative citation, Putman et al. / Shefchek et al.]
- [18] [TBD: Biolink Model citation, Unni et al., *Clinical and Translational Science*]